Pålitelig transportprotokoll (DRTP)

Data 2410

Kandidatnummer: 201

Introduksjon av prosjekt

Innen datanettverk nevnes ofte to protokoller når det snakkes om transportlaget: UDP og TCP. TCP er en forbindelsesorientert transportprotokoll som garanterer for at datapakker kommer frem til mottaker i riktig rekkefølge. UDP, derimot, er en forbindelsesløs og upålitelig transportprotokoll. Det opprettes ingen forbindelse med UDP, og protokollen kan heller ikke garantere for at data kommer frem i riktig rekkefølge, eller at data i det hele tatt når frem til mottaker.

I dette prosjektet skal det utvikles en egen transportprotokoll kalt Data2410 Reliable Transport Protocol, eller bare DRTP. Dette skal være en enkel, pålitelig dataoverføringsprotokoll som skal gi pålitelig datalevering på toppen av UDP. Den skal med andre ord sikre at data som sendes kommer fram til mottaker, og at data som sendes ankommer mottaker i riktig rekkefølge.

Det skal lages et program som sikrer pålitelig dataoverføring mellom en klient og en server av datapakker på maks 1000 bytes, hvor hver datapakke skal ha en header på nøyaktig 6 bytes. Data skal sendes fra klient til server. Påliteligheten skal sikres ved implementering av "Three Way Handshake" og "Go-Back-N".

Implementasjonen av "Three Way Handshake" skal sikre at det opprettes en forbindelse mellom klient og server før data overføres. Prosedyren går ut på å opprette en forbindelse etter en utveksling av tre meldinger: en synkroniseringsmelding fra klient til server for å initiere forbindelsen, en synkroniserings- og bekreftelsesmelding tilbake til klient fra server, og til slutt en bekreftelsesmelding tilbake til server fra klienten, derav navnet "Three Way Handshake"

Når forbindelsen er opprettet, kan dataoverføringen starte. Det er her implementasjonen av "Go-Back-N" kommer inn i bildet. GBN skal sørge for at datapakker ikke går tapt, og at alle datapakker leveres i den rekkefølgen de ble sendt. Går en pakke tapt, eller ankommer mottaker i feil rekkefølge, skal klienten gjenoppta sendingen fra siste pakken som ble bekreftet mottatt av server. Prosedyren er relativt simpel. Klienten sender pakker med data til server med spesifikke sekvensnummer, og mottar bekreftelsesmeldinger på at pakkene med de gitte sekvensnumrene er mottatt av server i retur. N i Go-Back-N refererer til størrelsen på "sliding window". Dette vinduets størrelse gir antall pakker som kan sendes til serveren før klienten har fått en bekreftelsesmelding i retur.

Til slutt skal det implementeres en prosedyre for lukking av forbindelsen på en kontrollert måte. Dette er en enkel "Two Way handshake" hvor klienten bekrefter at overføringen er ferdig ved å sende en FIN-pakke. I retur mottar klienten en FIN-pakke som også bekrefter at server mottok FIN-pakken sendt fra klienten.

Implementasjon av DRTP

Prosjektet består av to Python filer: application.py og functions.py. Filen functions.py inneholder kun funksjoner som benyttes av application.py. Filen application.py er selve programmet, og inneholder relativt lite kode sammenlignet med functions.py. Dette gjorde jeg for at application.py skulle bli så ryddig som mulig. Om man ser bort fra funksjonen argumentHåndtering (som håndterer kommandolinjeargumentene), inneholder den kun en main-funksjon hvor det mer eller mindre bare gjøres kall på det som kan sies å være hovedfunksjonene som utgjør applikasjonen: threeWayHandshake og GBN. Jeg skal nå gå litt nærmere inn på hvordan disse to funksjonene fungerer

Funksjonen threeWayHandshake

Denne funksjonen oppretter forbindelsen mellom klient og server ved hjelp av UDP sockets, og har en klientdel og en serverdel. Tanken var først å lage en egen threeWayHandshake-funksjon for både server og klient, men det ble lettere for min del å ha kun ha en funksjon som håndterte både server og klient. Her er min implementasjon av klientdelen av funksjonen:

```
# ThreeWayHandshake sin klientkode:
if args.client:
    serverip = args.ip                  # Henter server sin IP-adresse
    serverport = args.port              # Henter server sitt portnummer
    flagNr = 8                                  # Syn-flagget er satt. 1 0 0 0 er 8 som desimaltall
    antall = 1                                  # Antall forsøk på å koble til server
    print("Connection Establish Phase:\n")
    klientsocket = socket(AF_INET, SOCK_DGRAM)          # Oppretter UDP socket (klient)
    klientsocket.sendto(opprettePakke(seqNrKlient, ackNrKlient, flagNr, b''), (serverip, serverport))      # Sender SYN-pakke til server
    print("SYN packet is sent")
    klientsocket.settimeout(0.5)                    # Setter timeout til 0.5 sekunder
    while True:
        try:
            melding, serveradresse = klientsocket.recvfrom(2048)            # Pakke mottatt fra server
            header, data = parsePakke(melding)                              # Pakker ut pakken fra server
            serverSeq, serverAck, serverFlags = header                      # Henter ut verdiene fra pakkeheader
            syn, ack, fin = parseFlags(serverFlags)                         # Henter ut flaggene
            if syn and ack and not fin:
                seqNrKlient += 1                # Øker sequence number til server med 1 for neste pakken som skal sendes (en ACK)
                flagNr = 4                      # Ack-flagget satt. 0 1 0 0 blir 4
                print("SYN-ACK packet is received")
                klientsocket.sendto(opprettePakke(seqNrKlient, serverSeq, flagNr, b''), serveradresse)
                print("ACK packet is sent")
                print("Connection established\n")
                return klientsocket, serveradresse, seqNrKlient, ackNrKlient
        except TimeoutError:
            klientsocket.sendto(opprettePakke(seqNrKlient, ackNrKlient, flagNr, b''), (serverip, serverport))
            antall += 1                             # Øker antall retransmissions med 1
            if antall >= 4:                         # Prøver å koble til server et gitt antall ganger før den den gir seg
                print("\nConnection failed")
                return None
```

Denne koden vil kun utføres om man kjører application.py i klientmodus.
Klientdelen er implementert slik at den skal prøve å koble seg til serveren et gitt antall ganger ved å sende en SYN-pakke til server. Om dette ikke lykkes, enten fordi server ikke kjører, IP-adressen og/eller portnummer er feil, eller om SYN-pakken går tapt på vei til server, skal det skrives en melding til skjerm om at tilkoblingen mislyktes og det returneres None. Klienten vil vente i 0.5 sekunder på respons før den sender et nytt forsøk (timeout satt til 0.5).
Om SYN-pakken når frem til serveren, skal klienten motta en SYN-ACK-pakke i retur. Da vil koden i if-setningen som sjekker for om syn og ack er True utføres. Klienten vil da oppdatere variabelen flagNr slik at det sendes en ACK-pakke i retur til server. I dette tilfellet vil klienten tolke det som en vellykket tilkobling til server, og en tuple bestående av en tilkoblingssocket, servers IP-adresse og portnummer samt sekvensnummer og acknowledgement-nummer som skal settes på neste pakke fra klienten, vil returneres. Grunnen til at alt dette returneres er at dette skal brukes av GBN-funksjonen videre.

Funksjonene opprettePakke, parsePakke og parseFlags brukes henholdsvis til å opprette en pakke som skal sendes, pakke ut en pakke som er mottatt (i header og data) og til å hente ut hvert flagg i den mottatte pakken.

På serversiden ser threeWayHandshake slik ut:

```
# ThreeWayHandshake sin serverkode:
if args.server:
    flagNr = 12                                     # Syn og ack satt. 1 1 0 0 blir 12
    serversocket = socket(AF_INET, SOCK_DGRAM)      # Oppretter UDP socket (server)
    try:
        serversocket.bind((args.ip, args.port))     # Prøver å binde socketen til den gitte IP-adressen og portnummeret
    except OSError as e:
        print("Kunne ikke binde socket: " + str(e))
        return None
    melding, klientadresse = serversocket.recvfrom(2048)            # Pakke mottatt fra klient
    header, data = parsePakke(melding)                              # Pakker ut pakken fra klient
    klientSeq, klientAck, klientFlags = header                      # Henter ut verdiene fra pakkeheader
    ackNrServer = klientSeq                                         # Setter serveren sin Acknowledgement number lik mottatt sequence number fra klient
    syn, ack, fin = parseFlags(klientFlags)                         # Henter ut flaggene
    if not(ack or fin) and syn:
        print("SYN packet is received")
        serversocket.sendto(opprettePakke(seqNrServer, ackNrServer, flagNr, b''), klientadresse)
        print("SYN-ACK packet is sent")
        serversocket.settimeout(0.5)   # Setter en timeout slik at det returneres None om ikke en ACK fra klient kommer fram til server
        try:
            # Samme som kommenter over:
            melding, klientadresse = serversocket.recvfrom(2048)
            header, data = parsePakke(melding)
            klientSeq, klientAck, klientFlags = header
            ackNrServer = klientSeq
            syn, ack, fin = parseFlags(klientFlags)
            if not(syn or fin) and ack:
                print("ACK packet is received")
                print("Connection established")
                return serversocket, klientadresse, seqNrServer
            else:
                return None                 # Returnerer None om ACK-flagget ikke er satt i mottatt pakke
        except TimeoutError:
                return None                 # Returnerer None om det ikke er mottatt en ACK-pakke inne tidsrammen
```

Denne koden vil kun utføres om man kjører application.py i servermodus.
Her prøves det først å binde serverens UDP socket til en spesifikk IP-adresse og portnummer. Det kan være IP-adresse og/eller portnummer gitt fra kommandolinjen, eller standard IP-adresse (10.0.1.2) og/eller portnummer (8088). Om ikke dette lykkes, vil du få en beskjed til skjerm om at man ikke kunne binde socket, og serveren avsluttes ved at det returneres None.
Om det derimot lykkes, vil serveren lytte til klientforespørsler ved å vente på en innkommende SYN-pakke.
Når en SYN-Pakke er mottatt, vil koden i if-setningen som sjekker om den mottatte pakken hadde SYN-flagget satt utføres. Da sendes det en SYN-ACK-pakke tilbake til server, og en timer settes i gang (satt til 0.5s). Nå vil server forvente å få en ACK-pakke i retur, og om den ikke får det innen den satte tidsrammen, vil tilkoblingen til klient feile ved at det returneres None. Hvis den derimot mottar en ACK-pakke fra klienten innen tidsrammen, har tilkoblingen lyktes. Da returneres en tuple med tilkoblingssocket, klientens IP-adresse og portnummer samt sekvensnummer som skal brukes på neste pakke til server.
Funksjonene opprettePakke, parsePakke og parseFlags brukes på samme måte på serversiden som på klientsiden.

Funksjonen GBN

Denne funksjonen sørger for den pålitelige overføring av data mellom klient og server, og er, i likhet med threeWayHandshake, delt opp i en klient- og serverdel. Den sørger også for at tilkoblingen avsluttes på en kontrollert måte når dataoverføringen er fullført.

På klientsiden ser GBN slik ut:

```python
# GBN sin klientkode:
if args.client:
    print("Data Transfer:\n")
    datablokker = dataliste                 # Filen som skal overføres til klient delt opp i en liste med datablokker
    tilkoblingssocket, serveradresse, seqNrKlient, ackNrKlient = resultatHandshake          # Resultatet fra threeWayHandshake
    flagNr = 4              # Setter ACK-flagget
    vinduStr = args.window  # Vindustørrelsen
    vindu = []              # sliding window
    antall = 0              # Variabel for antall retransmissions
    # Sender de første n antall pakker
    for i in range(vinduStr):
            tilkoblingssocket.sendto(opprettePakke(seqNrKlient, ackNrKlient, flagNr, datablokker[seqNrKlient-1]), serveradresse)
            vindu.append(seqNrKlient)               # Legger til sendte pakkers sequence number til sliding window
            print(f"{datetime.datetime.now().time()} -- packet with seq = {seqNrKlient} is sent, sliding window = {str(vindu).replace('[', '{').replace(']', '}')}")
            seqNrKlient += 1

    while seqNrKlient-1 < len(datablokker):
        try:
            melding, serveradresse = tilkoblingssocket.recvfrom(2048)
            header, data = parsePakke(melding)
            serverSeq, serverAck, serverFlags = header
            ackNrKlient = serverAck
            syn, ack, fin = parseFlags(serverFlags)
            if not(syn or fin) and ack and serverAck == vindu[0]:
                print(f"{datetime.datetime.now().time()} -- ACK for packet = {serverAck} is received")
                tilkoblingssocket.sendto(opprettePakke(seqNrKlient, serverSeq, flagNr, datablokker[seqNrKlient-1]), serveradresse)
                vindu.append(seqNrKlient)               # Legger til siste sendte pakke (sequence number) i sliding window
                vindu.pop(0)                            # Forskyver sliding window når ACK er mottatt for laveste sequence number i vinduet
                print(f"{datetime.datetime.now().time()} -- packet with seq = {seqNrKlient} is sent, sliding window = {str(vindu).replace('[', '{').replace(']', '}')}")
                seqNrKlient += 1
                antall = 0          # Tilbakestiller antall retransmissions
                # Koden under tar seg av de siste ACK-pakkene når det ikke er flere pakker å sende fra klienten:
                if seqNrKlient-1 == len(datablokker):
                    while vindu:
                        try:
                            melding, serveradresse = tilkoblingssocket.recvfrom(2048)
                            header, data = parsePakke(melding)
                            serverSeq, serverAck, serverFlags = header
                            syn, ack, fin = parseFlags(serverFlags)
                            if not(syn or fin) and ack:
                                print(f"{datetime.datetime.now().time()} -- ACK for packet = {serverAck} is received")
                                vindu.pop(0)
                                antall = 0
                        except TimeoutError:
                            # Kode for retransmissions:
                            antall += 1
                            if antall > 4:       # Setter et maks antall retransmissions. Sikrer at klienten ikke havner i en uendelig løkke om serveren avsluttes
                                sys.exit()
                            else:
                                print(f"{datetime.datetime.now().time()} -- RTO occured")
                                # Sender alle pakkene i det nåværende sliding window:
                                for i in vindu:
                                    tilkoblingssocket.sendto(opprettePakke(i, ackNrKlient, flagNr, datablokker[i-1]), serveradresse)
                                    print(f"{datetime.datetime.now().time()} -- retransmitting packet with seq = {i}")
        except TimeoutError:
            # Kode for retransmissions:
            antall += 1
            if antall > 4:          # Setter et maks antall retransmissions. Sikrer at klienten ikke havner i en uendelig løkke om serveren avsluttes
                sys.exit()
            else:
                print(f"{datetime.datetime.now().time()} -- RTO occured")
                # Sender alle pakkene i det nåværende sliding window:
                for i in vindu:
                    tilkoblingssocket.sendto(opprettePakke(i, ackNrKlient, flagNr, datablokker[i-1]), serveradresse)
                    print(f"{datetime.datetime.now().time()} -- retransmitting packet with seq = {i}")
```

```python
#teardown klient
print("DATA Finished\n\n")
print("Connection Teardown:\n")
flagNr = 2                        # FIN-flagget satt (0 0 1 0 blir 2)
tilkoblingssocket.sendto(opprettePakke(seqNrKlient, ackNrKlient, flagNr, b''), serveradresse)
print("FIN packet packet is sent")
antall = 0          # Variabel for antall forsøk på å sende FIN til server
while True:
    try:
        melding, klientadresse = tilkoblingssocket.recvfrom(2048)
        header, data = parsePakke(melding)
        serverSeq, serverAck, serverFlags = header
        syn, ack, fin = parseFlags(serverFlags)
        if fin and ack and not syn:             # Avslutter om FIN-ACK mottas fra server
            print("FIN ACK packet is received")
            print("Connection Closes")
            tilkoblingssocket.close()
            break
    except TimeoutError:
        tilkoblingssocket.sendto(opprettePakke(seqNrKlient, ackNrKlient, flagNr, b''), serveradresse)
        antall += 1
        if antall >= 4:         # Slutter å sende FIN til server etter et gitt antall ganger
            tilkoblingssocket.close()
            break
```

På klientsiden tar GBN inn tre ting: resultatet fra threeWayHandshake, en dataliste bestående av byte-strenger på 994 bytes av filen som skal sendes, og kommandolinjeargumentene.
Først starter klienten med å sende de n første (window size) pakkene i datalisten jeg nettopp nevnte, og legger alle sekvensnumrene som er sendt inn i listen "vindu". Deretter vil den gå inn i en while-løkke.
I while-løkken vil klienten vente på å motta ACK-pakker fra server på tidligere sendte pakker, og sende nye pakker til server når ACK-pakker er mottatt. Koden i løkken fungerer som følger:

- Om en ACK-pakke med Acknowledgement-nummer lik sekvensnummeret til første pakke sendt fra klient ankommer klienten innen tidsrammen på 0.5 sekunder, sendes pakke nummer n+1, og "vindu" (vindu.append(seqNrKlient) og vindu.pop(0) i koden) og sekvensnummer oppdateres. Hvis dette ikke er tilfellet, antas det at den første pakken aldri kom fram til server. Da sendes pakkene med sekvensnumre i listen "vindu" på nytt.
- Når ACK-pakken for den første pakken sendt til server er mottatt, ventes det på ACK for den andre pakken som ble sendt. Mottas ACK-pakke med Acknowledgement-nummer lik sekvensnummeret til andre pakke som ble sendt fra klient innen tidsrammen, sendes pakke nummer n+2, og "vindu" og sekvensnummer oppdateres. Hvis ikke, sendes alle pakker med sekvensnummer fra den siste mottatte Acknowledgement-nummer fra server og opp til siste sendte sekvensnummer. Altså pakker med sekvensnumre i listen "vindu".

Og sånn fortsetter det for pakke nummer n+3, n+4, n+5 osv. Helt til siste pakke er sendt.
Når siste pakke er sendt, vil if-setningen med betingelsen seqNrKlient == len(datablokker) bli sann. Da vil server bare vente på å motta, og eventuelt re-sende, de siste ACK-pakkene som ikke ennå er mottatt fra server.
Når siste ACK-pakke fra server er mottatt, vil klienten gå inn i avslutningsdelen. Da sendes det en FIN-pakke til server, og tilkoblingssocket vil stenges når enten klienten har mottatt en FIN-ACK-pakke i retur, eller når antall forsøk på å re-sende en FIN-pakke uten noe svar tilbake fra server overskrider et gitt antall (if antall >= 4 i koden. 4 er bare valgt tilfeldig).
Legg også merke til at antall re-sendinger (retransmissions) også er begrenset til et gitt antall ganger i selve dataoverføringen. Dette gjøres for å unngå at klienten re-sender pakker i det uendelige om serveren skulle krasje under sending, eller om server aldri mottok ACK-pakken fra klienten under three way handshake.

På serversiden ser GBN slik ut:

```python
# GBN sin serverkode:
if args.server:
    tilkoblingssocket, klientadresse, seqNrServer = resultatHandshake
    flagNr = 4                                  # Setter ack-flagget slik at pakker fra server blir ack-pakker
    forventetSeqnr = 1                          # Variabel for forventet sekvensnummer. Brukes for å se om pakker ankommer i riktig rekkefølge
    pakkerMottatt = []                          # Liste som inneholder alle pakker mottatt fra klient. Brukes til å beregne Throughput
    tilkoblingssocket.settimeout(None)          # Setter timeout tilbake til None fordi timeout ble satt til 0.5s på serversiden i ThreeWayHandshake
    startid = datetime.datetime.now()           # Tiden hvor overføring av data starter
    while True:                                 # Løkke som går helt til server mottar FIN
        melding, klientadresse = tilkoblingssocket.recvfrom(2048)
        pakkerMottatt.append(melding)
        header, data = parsePakke(melding)
        klientSeq, klientAck, klientFlags = header
        syn, ack, fin = parseFlags(klientFlags)
        if not(syn or ack) and fin:
            # teardown server
            sluttid = datetime.datetime.now()           # Tiden hvor overføring av data er fullført
            flagNr = 6                                  # Setter flaggene FIN og ACK (0 1 1 0 blir 6)
            print("FIN packet is received")
            tilkoblingssocket.sendto(opprettePakke(seqNrServer, klientSeq, flagNr, b''), klientadresse)
            print("FIN ACK packet is sent\n")
            print(f"The throughput is {round(throughput(startid, sluttid, pakkerMottatt), 3)} Mbps")
            print("Connection Closes")
            tilkoblingssocket.close()
            break
        elif args.discard is not None and args.discard == klientSeq:   # Sender ikke ACK tilbake til server om pakken fra server har sequence number lik discard-verdi
            args.discard = None         # Setter discard-verdi til None for at ikke server skal forkaste pakken med et spesifikk sequence number hver gang.
            continue
        elif forventetSeqnr < klientSeq:            # Håndterer pakker som ikke ankommer i rekkefølge
            print(f"{datetime.datetime.now().time()} -- out-of-order packet {klientSeq} is received")
        elif forventetSeqnr > klientSeq:            # Håndterer duplikater ved å sende ACK for pakken på nytt
            tilkoblingssocket.sendto(opprettePakke(seqNrServer, klientSeq, flagNr, b''), klientadresse)
        else:                                       # Håndterer pakker som ankommer i riktig rekkefølge
            print(f"{datetime.datetime.now().time()} -- packet {klientSeq} is received")
            print(f"{datetime.datetime.now().time()} -- sending ack for the received {klientSeq}")
            lagreFil("bilde.jpg", data)             # Lagrer data som ankommer i riktig rekkefølge
            tilkoblingssocket.sendto(opprettePakke(seqNrServer, klientSeq, flagNr, b''), klientadresse)
            forventetSeqnr += 1
```

På serversiden tar GBN inn resultatet fra threeWayHandshake og kommandolinjeargumentene. Når forbindelsen med klienten er etablert, vil serveren vente på pakker fra klienten (while-løkken). Her vil sekvensnummeret og flagget som er satt på pakken som mottas avgjøre hva serveren foretar seg. Variabelen "forventetSeqnr" brukes for å påse at det kun er pakker i riktig rekkefølge som aksepteres. Avhengig av pakkens sekvensnummer og flagget som er satt på pakken, vil server gjøre følgende:

- Om mottatt pakke har sekvensnummer lik forventetSeqnr og ACK-flagget er satt, vil server lagre data mottatt i en fil kalt "bilde.jpg" ved kall på funksjonen lagreFil, sende en ACK-pakke men Acknowledgement-nummer lik mottatt sekvensnummer til server og øke forventetSeqnr med 1.
- Om mottatt pakke har sekvensnummer mindre enn forventetSeqnr, vil det sendes en ACK-pakke tilbake til server med Acknowledgement-nummer lik mottatt sekvensnummer, men ingen data lagres. Grunnen er at server allerede har mottatt denne pakken og lagret tilhørende data tidligere (duplikat).
- Om mottatt pakke har sekvensnummer større enn forventetSeqnr, skal det kun skrives en beskjed til skjerm om at det er mottatt en pakke ute av rekkefølge, men ikke sendes noe tilbake til server. Dette vil føre til at server gjenopptar sending av pakkene i sitt vindu (sliding window), slik at pakker vil ankomme i riktig rekkefølge igjen.
- Om mottatt pakke har sekvensnummer lik discard-verdien man kan gi som kommandolinjeargument, skal server ikke foreta seg noen ting, bare ignorere pakken én gang. Dette gjøres for å kunne teste om server sender pakker på nytt (retransmission) når den ikke mottar en ACK-pakke.
- Om mottatt pakke er en FIN-pakke (FIN-flagg satt), betyr det at dataoverføringen er ferdig. Da skal server sende en FIN-ACK-pakke i retur, stenge tilkoblingssocket og hoppe ut av while-løkken.

Beskjed om mulig feiltolkning av oppgaven

En ting jeg bør nevne til slutt om min GBN er at i denne implementasjonen av GBN vil Acknowledgement-nummer være lik sekvensnummeret på forrige pakke mottatt, ikke det forventede sekvensnummeret på neste pakke. Jeg kan ha tolket oppgaven feil, men jeg tolket den slik at sånn jeg har gjort det var sånn man skulle gjøre det i denne oppgaven. Om jeg har tolket feil, så skulle det vært slik at forventetSeqNr-variabelen skulle vært oppdatert før ACK-pakken ble sendt til server i denne kodebiten:

```
else:                                   # Håndterer pakker som ankommer i riktig rekkefølge
    print(f"{datetime.datetime.now().time()} -- packet {klientSeq} is received")
    print(f"{datetime.datetime.now().time()} -- sending ack for the received {klientSeq}")
    lagreFil("bilde.jpg", data)         # Lagrer data som ankommer i riktig rekkefølge
    tilkoblingssocket.sendto(opprettePakke(seqNrServer, klientSeq, flagNr, b''), klientadresse)
    forventetSeqnr += 1
```

Acknowledgement-nummeret sendt til klienten skulle da vært forventetSeqNr, ikke klienSeq.
På klientsiden skulle da den første if-setningen i while-løkken i GBN ha hatt serverAck == vindu[0] + 1 istedenfor serverAck == vindu[0], utskriften "ACK for pakket = {serverAck} is received" skulle da vært utskriften "ACK for pakket = {serverAck-1} is received", og Acknowledgement-nummeret på pakkene som klienten sender til server skulle da vært 1.
Har jeg tolket oppgaven riktig, kan man bare se bort fra denne beskjeden.

Main-funksjonen i application.py

```
def main():
    """
    Beskrivelse:
    Main-funksjonen utfører selve programmet: Pålitelig overføring av data over UDP.
    """
    args = argumenthåndtering()                                     # Henter kommandolinjeargumentene
    resultatHandshake = threeWayHandshake(args)                     # Kaller og Henter ut resultatet av threeWayhandshake
    # Klientsiden:
    if args.client:
        dataliste = hentFil(args.file)
        if resultatHandshake is not None:
            GBN(resultatHandshake, dataliste, args)         # Starter dataoverføring om forbindelsen er opprettet
        else:
            sys.exit()                                              # Avslutter om threeWayHandshake returnerer None (Ingen forbindelse)
    # Serversiden
    if args.server:
        if resultatHandshake is not None:
            GBN(resultatHandshake, None, args)              # Starter dataoverføring om forbindelsen er opprettet
        else:
            sys.exit()                                              # Avslutter om threeWayHandshake returnerer None (Ingen forbindelse)

if __name__ == "__main__":
    main()
```

Main-funksjonen i application.py er ganske simpel. Det gjøres stort sett bare kall på threeWayHandshake og GBN. Om threeWayHandshake returnerer None, vil programmet avslutte. Dette gjøres fordi det da ikke er opprettet en forbindelse mellom klient og server. Da skal programmet avsluttes.
På klientsiden kalles det en funksjon kalt hentFil. Denne funksjonen returnerer filen som skal sendes til server i en liste bestående av byte-strenger på 994 bytes.
En siste ting å nevne kan være at funksjonen argumenthåndtering kalles i main. Denne funksjonen kaller igjen en annen funksjon kalt verdihåndtering som returnerer True om kommandolinjeargumentene oppfyller noen kriterier (som f.eks. formatet til IP-adressen og portnummer), ellers False. Funksjonen argumenthåndtering vil returnere et objekt med kommandolinjeargumentene om verdihåndtering returnerer True, eller avslutte programmet om verdihåndtering returnerer False.

Diskusjoner

1. Vindusstørrelse og gjennomstrømning (throughput)

Etter å ha kjørt application.py med vindusstørrelse 3, 5 og 10, fikk jeg disse resultatene:
For vindusstørrelse = 3:

```
21:46:14.233297 -- sending ack for the received 1837
FIN packet is received
FIN ACK packet is sent

The throughput is 0.237 Mbps
Connection Closes
```

For vindusstørrelse = 5:

```
21:58:29.314611 -- sending ack for the received 1837
FIN packet is received
FIN ACK packet is sent

The throughput is 0.395 Mbps
Connection Closes
```

For vindusstørrelse = 10:

```
21:59:28.887903 -- sending ack for the received 1837
FIN packet is received
FIN ACK packet is sent

The throughput is 0.788 Mbps
Connection Closes
```

Vi ser av resultatene over at vi får en høyere throughput jo høyere vindusstørrelsen er. For å prøve å forstå dette bedre kan vi først se på hva som skjer om vindusstørrelsen er lik 1.
Med en vindusstørrelse lik 1 kan vi kun sende én pakke av gangen før vi har mottatt en bekreftelse fra mottaker. Det vil altså kun være 1 pakke i omløp av gangen til hver tid.
Øker vi vindusstørrelsen til 3, vil vi kunne sende tre pakker før man har mottatt noen bekreftelse. I dette tilfellet vil det da være 3 pakker i omløp av gangen til hver tid. Dermed får vi sendt flere pakker i løpet av en gitt tid med vindusstørrelse lik 3 enn hva tilfellet er med vindusstørrelse lik 1.
Øker vi vindusstørrelsen videre til 5, vil vi få 5 pakker i omløp til en hver tid, og dermed også sendt flere pakker i løpet av en gitt tid enn hva tilfellet er ved vindusstørrelse lik 3.
Og det samme vil være tilfellet med vindusstørrelse 10 og 5: Ved vindusstørrelse lik 10 blir det sendt flere pakker i løpet av en gitt tid enn ved vindusstørrelse lik 5, faktisk dobbelt så mange.

Siden throughput er et mål for hvor mye data som er ankommet mottaker per tid, skjønner man hvorfor throughput blir høyere jo høyere vindusstørrelsen er: mer data sendes over kortere tid.
Så da kan man vel kanskje konkludere med at så lenge vindusstørrelsen ikke blir så stor at den overgår nettverkskapasiteten, vil en høy vindusstørrelse være ønskelig.

2. Vindusstørrelse, gjennomstrømning (throughput) og RTT

Vi så tidligere at vindusstørrelsen påvirker throughput. Men hvordan vil ulike verdier for RTT påvirke throughput for de samme vindusstørrelsene?
Tabellen nedenfor viser throughput for forskjellige RTT for de samme vindusstørrelsene som vi så på tidligere:

	RTT = 50ms	**RTT = 100ms**	**RTT = 200ms**
Vindusstørrelse = 3	0.471 Mbps	0.237 Mbps	0.119 Mbps
Vindusstørrelse = 5	0.784 Mbps	0.395 Mbps	0.197 Mbps
Vindusstørrelse = 10	1.57 Mbps	0.788 Mbps	0.394 Mbps

Av tabellen over ser vi at det fortsatt er sånn at vi får en høyere throughput jo høyere vindusstørrelsen er for hver RTT. Men vi kan også se at jo høyere RTT blir, jo lavere blir throughput for hver vindusstørrelse. Det ser faktisk ut til at throughput halveres for hver vindusstørrelse når RTT dobles.

Om man tenker på RTT som tiden det tar for en melding å gå fra sender til mottaker og tilbake igjen, gir jo dette egentlig mening. I min applikasjon vil RTT bli den tiden det tar for en pakke å gå fra sender til mottaker pluss tiden det tar før en ACK-pakke har kommet tilbake. Hvis denne tiden dobles, gir det jo også mening at den tiden det tar for den totale mengden data å overføres blir dobbelt så lang, og dermed halvert throughput. Eller motsatt: Om RTT halveres, halveres den totale tiden, og dermed dobles throughput.

En interessant ting å legge merke til er at throughput for lavere vindusstørrelser kan bli en god del høyere enn throughput for høyere vindusstørrelser om RTT er mye lavere. Med RTT= 50ms vil selv en vindusstørrelse på bare 3 gi en greit bedre throughput enn hva en RTT= 200ms og en vindusstørrelse på 10 vil gi.

3. Forkaste pakker med --discard

I bildene under er server og klient kjørt slik:

- python3 application.py -c -i 10.0.1.2 -p 8088 -f iceland_safiqul.jpg -w 5
- python3 application.py -s -i 10.0.1.2 -d 1800

Klient:

```
18:07:42.088725 -- ACK for packet = 1798 is received
18:07:42.088744 -- packet with seq = 1803 is sent, sliding window = {1799, 1800, 1801, 1802, 1803}
18:07:42.088759 -- ACK for packet = 1799 is received
18:07:42.088774 -- packet with seq = 1804 is sent, sliding window = {1800, 1801, 1802, 1803, 1804}
18:07:42.590413 -- RTO occured
18:07:42.590665 -- retransmitting packet with seq = 1800
18:07:42.590739 -- retransmitting packet with seq = 1801
18:07:42.590783 -- retransmitting packet with seq = 1802
18:07:42.590820 -- retransmitting packet with seq = 1803
18:07:42.590841 -- retransmitting packet with seq = 1804
18:07:42.692147 -- ACK for packet = 1800 is received
18:07:42.692315 -- packet with seq = 1805 is sent, sliding window = {1801, 1802, 1803, 1804, 1805}
18:07:42.692788 -- ACK for packet = 1801 is received
18:07:42.692875 -- packet with seq = 1806 is sent, sliding window = {1802, 1803, 1804, 1805, 1806}
18:07:42.692923 -- ACK for packet = 1802 is received
18:07:42.692962 -- packet with seq = 1807 is sent, sliding window = {1803, 1804, 1805, 1806, 1807}
18:07:42.692993 -- ACK for packet = 1803 is received
18:07:42.693020 -- packet with seq = 1808 is sent, sliding window = {1804, 1805, 1806, 1807, 1808}
18:07:42.693049 -- ACK for packet = 1804 is received
18:07:42.693089 -- packet with seq = 1809 is sent, sliding window = {1805, 1806, 1807, 1808, 1809}
18:07:42.793135 -- ACK for packet = 1805 is received
```

Server:

```
18:07:42.088522 -- sending ack for the received 1797
18:07:42.088584 -- packet 1798 is received
18:07:42.088596 -- sending ack for the received 1798
18:07:42.088630 -- packet 1799 is received
18:07:42.088640 -- sending ack for the received 1799
18:07:42.189337 -- out-of-order packet 1801 is received
18:07:42.189456 -- out-of-order packet 1802 is received
18:07:42.189476 -- out-of-order packet 1803 is received
18:07:42.189539 -- out-of-order packet 1804 is received
18:07:42.691151 -- packet 1800 is received
18:07:42.691297 -- sending ack for the received 1800
18:07:42.692101 -- packet 1801 is received
18:07:42.692175 -- sending ack for the received 1801
18:07:42.692579 -- packet 1802 is received
18:07:42.692605 -- sending ack for the received 1802
18:07:42.692737 -- packet 1803 is received
18:07:42.692758 -- sending ack for the received 1803
18:07:42.692888 -- packet 1804 is received
18:07:42.692906 -- sending ack for the received 1804
18:07:42.792832 -- packet 1805 is received
18:07:42.792900 -- sending ack for the received 1805
```

Her ser vi hvordan den pålitelige protokollen håndterer tap av pakker og pakker som ankommer mottaker i feil rekkefølge. Pakke nummer 1800 skal i dette tilfellet ignoreres av server én gang. Det vil si at den ikke vil ta imot pakken med sekvensnummer 1800 første gangen den mottas.
På serversiden ser vi da at pakke nummer 1801, 1802, 1803 og 1804 ankommer server ute av rekkefølge. Dette er pakkene/sekvensnumrene i klientens sliding window, altså de pakkene som er sendt uten å blitt bekreftet mottatt av server ennå.
På klientsiden ser vi at vi får en timeout (RTO occured) etter at ACK for pakke nummer 1799 er mottatt. Det betyr at klienten ikke mottok noen ACK for 1800 (i hvert fall i tide), så pakken med sekvensnummer 1800 antas å være tapt. Da sendes alle pakkene som ble sendt til server fra den

siste pakken som ble bekreftet mottatt av server (i dette tilfellet 1799) på nytt. Det er pakkene fra og med 1800 til og med 1804.
Går vi tilbake til serversiden igjen, ser vi at pakke nummer 1800 er ankommet, etterfulgt av de andre som tidligere ankom i feil rekkefølge. Rekkefølgen er nå opprettet igjen, og sending av nye pakker kan fortsette som tidligere.

4. Tap av pakker

Tap av pakker vil som vi så tidligere medføre sending av alle pakker fra siste pakke som er bekreftet mottatt av server til og med siste pakke sendt til server. Dette vil medføre mye re-sendinger om vi har mange pakketap. Det kan da være interessant å se på hvordan pakketap påvirker throughput.
I bildene under kan man se hva throughput ble med et pakketap på 2% og med et pakketap på 5%, begge med en vindusstørrelse på 3 og RTT=100ms:

2% pakketap

```
FIN packet is received
FIN ACK packet is sent

The throughput is 0.194 Mbps
Connection Closes
```

5% pakketap

```
FIN packet is received
FIN ACK packet is sent

The throughput is 0.145 Mbps
Connection Closes
```

Av bildene ser vi at man fikk en lavere throughput ved pakketap på 5% sammenlignet med 2%. Sammenligner vi disse verdiene med hvilken throughput man fikk for samme vindusstørrelse og RTT for 0 pakketap, 0.237 Mbps, ser det ut til at tendensen er at throughput minker når pakketapsprosenten øker. Reduksjonen er også ganske betydelig, særlig om vi ser på 0 pakketap og 5% pakketap. Dette er med andre ord merkbare forskjeller.

Men forskjellene gir jo også mening ettersom klienten vil bruke en del tid på å sende pakker på nytt til server. Det er jo heller ikke bare den tapte pakken som skal sendes på nytt, men også alle pakker sendt etter som ikke er bekreftet av server ennå. Dette tar tid og er ressurskrevende.

Så gitt en høy pakketapsrate, begynner protokollen (eventuelt implementeringen min eller en blanding) å vise tegn på at den fort kan bli mindre effektiv og mer ressurskrevende. Det var i hvert fall en merkbar forskjell i tiden det tok å fullføre overføringen av bildet når pakketapsprosenten økte.

Kanskje en protokoll hvor pakker som ankommer out-of-order etter et pakketap blir lagret hos mottaker, og hvor klient kun re-sender pakken som gikk tapt, ville vært mer effektiv når pakketapsraten er høy?

Kildeliste

- Islam, S (2024). *2410/header/header.py*. GitHub. https://github.com/safiqul/2410/blob/main/header/header.py
- Mine egne obligatoriske innleveringer i Data2410